

FMP

Fleet Management Platform

Fair · Real-Time · Multi-Role · Race-Condition-Safe Dispatch

CONTRIBUTORS

Ayush Tyagi

2023CSB1108

Aryan Singh

2023CSB1102

Nishant Sahni

2023CSB1140

Divyanshu K Verma

2023CSB1197

Live at: <https://fmp-dep.vercel.app/>

ASP.NET Core 8

Flutter / Dart

PostgreSQL 14+

SignalR

JWT + Google OAuth

ReportLab

1

Description

What FMP is and why it exists

FMP (Fleet Management Platform) is a logistics coordination system that connects cargo senders, drivers, fleet owners, and regulatory unions through a single mobile application. Built around **dispatch fairness** and **real-time synchronisation**, FMP replaces fragmented WhatsApp-based coordination with an automated, role-aware marketplace.

Built and tested on **Windows**. Requires: .NET 8.0 SDK · Flutter SDK ^3.10.7 · PostgreSQL 14+ · VS Code or Visual Studio 2022.

2

Problem & Approach

The fairness problem in Indian logistics

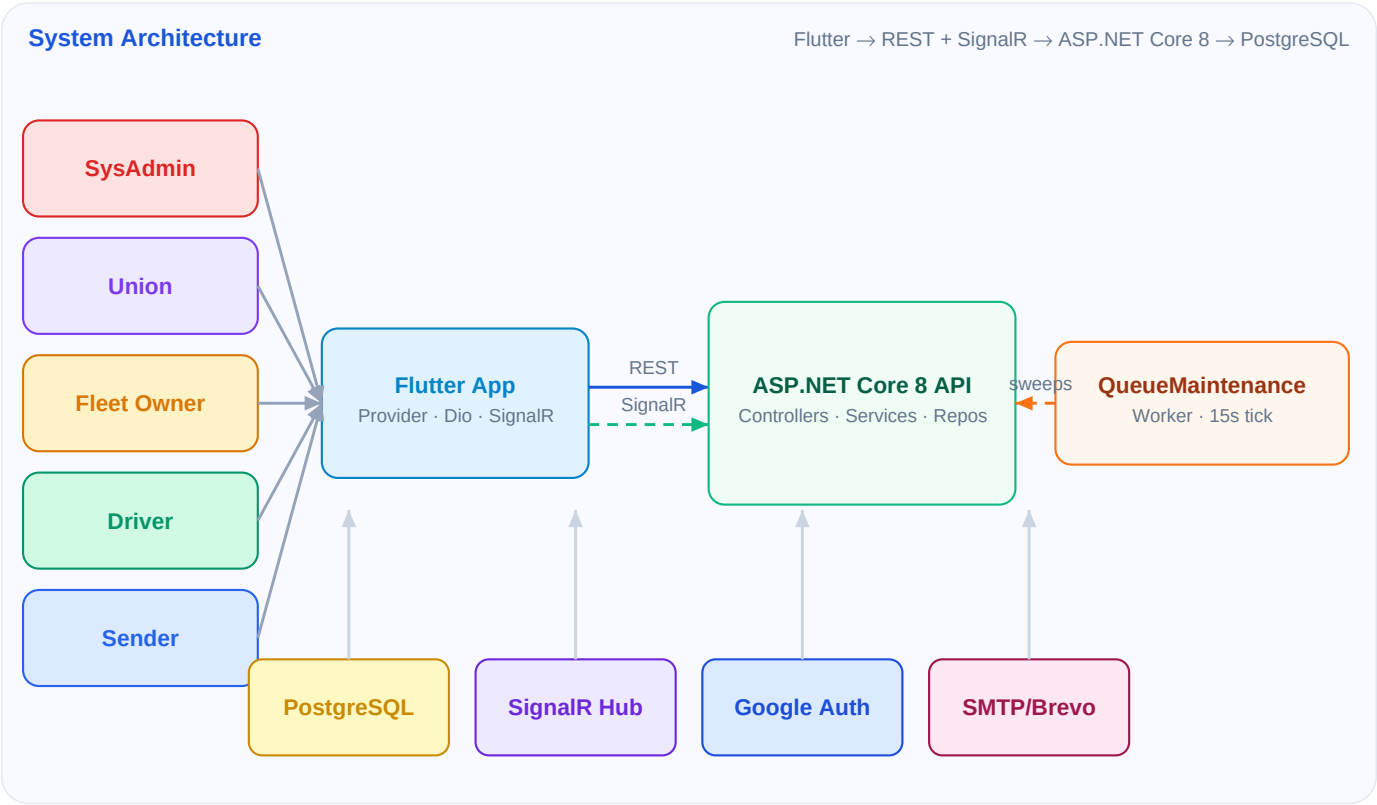
Indian logistics dispatching is typically a **race to the phone** — whoever responds first gets the job, regardless of experience or fairness. FMP solves this through a **Two-Tier Queue System**: time-slotted access windows ordered by driver experience (total trips completed) ensure every eligible driver gets a fair, sequential opportunity to claim shipments.

Problem	FMP Solution
First-come-first-served phone calls	Time-slotted windows by seniority (TotalTripsCompleted)
Race conditions on same shipment	SELECT FOR UPDATE SKIP LOCKED + 3-attempt retry
No real-time sync for all parties	SignalR broadcasts on every state change
Fragmented multi-app coordination	Single Flutter codebase, 5 roles, 1 install

3

Architecture

System components and data flow



All five client roles communicate through a single Flutter app over REST and SignalR. The ASP.NET Core 8 API hosts the business logic, delegates persistence to PostgreSQL, and spawns a background worker that ticks every 15 seconds to sweep expired queue assignments.

Component Breakdown

Component	Role	Technical Implementation
Flutter App	Single mobile codebase, all five roles	BloC/Provider · dio · signalr_netcore
ASP.NET Core 8 API	REST endpoints + SignalR hub	ShipmentQueueHub · JWT middleware · global exception handler
PostgreSQL	Primary datastore	ACID · FOR UPDATE SKIP LOCKED · triggers
QueueMaintenanceWorker	Background window sweep	BackgroundService · PeriodicTimer · 15 s
Google Auth	Identity verification	Google.Apis.Auth · ID Token flow
SMTP / Brevo	Transactional email	MailKit + Brevo REST API fallback

4

Tech Stack

Mobile Layer

Flutter / Dart

· Provider

· Dio

· SignalR

API Layer

ASP.NET Core 8

· Controllers

· Middleware

· JWT

Data Layer

PostgreSQL 14+

· ACID

· FOR UPDATE

· Triggers

Realtime

SignalR

· Push

· Broadcast

· Hub

Auth

JWT + Google OAuth

· Stateless

· Token

· Roles

Layer	Technology	Purpose
Backend	ASP.NET Core 8.0	REST API and business logic
Database	PostgreSQL 14+	Relational storage with ACID compliance

Layer	Technology	Purpose
Realtime	SignalR	Push notifications for queue & shipment updates
Mobile	Flutter / Dart	Cross-platform UI for all five roles
Auth	JWT + Google OAuth	Stateless authentication and identity
Worker	BackgroundService	Queue window expiry and event maintenance
Email	MailKit / Brevo	OTP delivery and transactional notifications

5

Role Overview

Five roles, one app

Sender



Creates shipments, manages cargo, tracks delivery.

Union



Reviews & approves shipments, gates queue entry.

Driver



Time-slotted offers, manages trips & billing.

Fleet Owner



Manages vehicles & drivers, monitors assignments.

SysAdmin



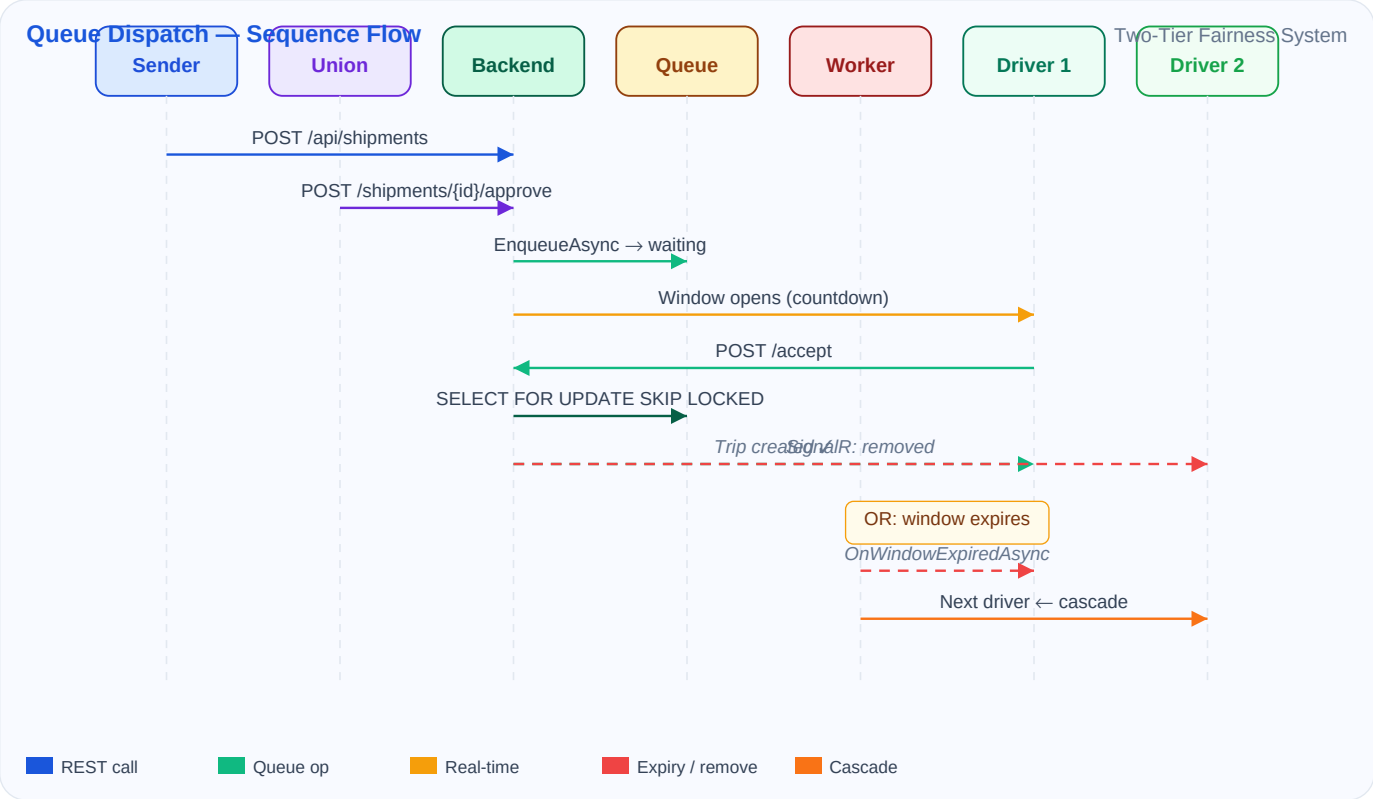
Metrics, user mgmt, manual dispatch overrides.

Role	Primary Capability
Sender	Creates shipments, manages cargo details, tracks delivery status
Union	Reviews and approves pending shipments, gates queue entry
Driver	Receives time-slotted shipment offers, manages active trips
Fleet Owner	Manages vehicles and drivers, monitors active assignments
SysAdmin	System metrics, user management, and manual dispatch overrides

6

The Queue System

Core innovation — Two-Tier Fairness Dispatch



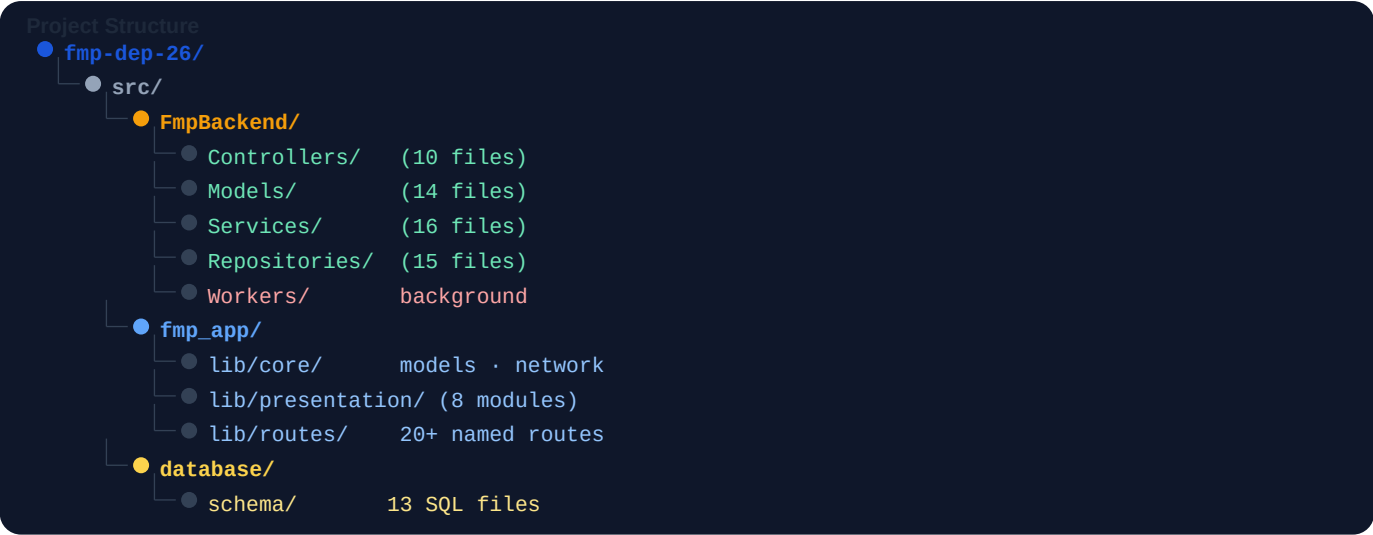
Tier 1 — ShipmentQueue (Live Dispatch Board)	Tier 2 — QueueEvent (Time-Slotted Access)
- Approved shipments enter with status=waiting - Race-condition safe via PostgreSQL FOR UPDATE SKIP LOCKED 3-attempt retry on concurrency conflicts	- Admin creates event → drivers get ordered positions (by trip count) - Each driver gets a per-shipment JSON list with countdown timers - Expiry cascades; accept removes from ALL other drivers' lists - Self-healing: finished drivers auto-reset to rejoin

Key Technical Properties

Property	Detail
Concurrency	FromSqlRaw executes SELECT ... FOR UPDATE SKIP LOCKED atomically
Seniority Ordering	Driver positions from TotalTripsCompleted in drivers table
Cascading Expiry	OnWindowExpiredAsync: marks slot → opens next shipment for same driver + next driver
Real-time Sync	SignalR broadcasts ShipmentAccepted to all connected clients immediately
Self-Healing	GetActiveEventForDriverAsync resets drivers who finished trips, rebuilds empty lists

7

Project Structure



Module	File Count	Key Files
Controllers	10	AuthController · ShipmentController · QueueEventsController · SysAdminController
Models	14	User · Driver · Shipment · ShipmentQueue · QueueEvent · Trip · SystemLog
Services	16	QueueEventService (663 ln) · ShipmentQueueService · OtpService · TripService
Repositories	15	ShipmentQueueRepository (FOR UPDATE) · DriverEligibleRepository · TripCrudRepository
Flutter Screens	~30	8 role modules · auth · onboarding · role_router · shared
SQL Scripts	13	00user → logs → updaters_and_views · trip_analytics_daily (materialized view)

8

Getting Started

Prerequisites

Requirement	Version	Notes
.NET SDK	8.0	Backend runtime
Flutter SDK	^3.10.7	Mobile app
PostgreSQL	14+	Primary datastore
VS Code / VS 2022	Latest	IDE

Database Setup — Run SQL scripts in order:

1. 00user.sql
2. 01roles.sql
3. 02union.sql
4. 03drivers.sql
5. trips.sql
6. support.sql
7. logs.sql
8. updaters_and_views.sql
9. insert_driver_vehicle_fleetmgr.sql

Run Backend

```
cd src/FmpBackend → dotnet run (API at http://localhost:5153)
```

Run Flutter App

```
cd src/fmp_app → flutter pub get → flutter run
```

Update `lib/core/network/api_client.dart` with your backend host if not running locally.

Environment Variables (appsettings.json)

Jwt.Key	Base64 encoded symmetric secret for HS256
Jwt.ExpiryMinutes	1440 (24 hours)
ConnectionStrings.Default	PostgreSQL connection string
Google.ClientId	Google OAuth Client ID for Flutter sign-in
Smtp.Host/Port/User/Pass	Gmail SMTP or Brevo relay credentials

9 API Overview

Controller	Route Prefix	Responsibility
AuthController	/auth	Email OTP, Google Sign-In, role resolution
ShipmentController	/api/shipments	Shipment CRUD, approval, rejection, search
QueueEventController	/api/queue-events	Queue event lifecycle, driver slot preview
ShipmentQueueController	/api/shipment-queue	Enqueued shipment status, accept, pass
TripsController	/api/trips	Trip creation, status updates, GPS tracking
DriverController	/drivers ■	Driver onboarding and fleet-owner lookups
VehiclesController	/vehicles ■	Vehicle CRUD and bulk management
SenderController	/senders	Sender / organisation onboarding
SysAdminController	/sysadmin	Metrics, logs, user management, force-assign

■ /drivers and /vehicles lack the /api/ prefix used by other routes — known inconsistency, tracked in Known Limitations.

10

Services Deep-Dive

16 business-logic files — line counts, key methods, responsibilities

All business logic lives in **src/FmpBackend/Services/**. Services are registered as **scoped** in Program.cs (30 scoped DI registrations + 1 singleton JwtService + 1 hosted QueueMaintenanceWorker). The most complex file is **QueueEventService.cs at 663 lines**, which owns the entire two-tier fairness system.

Service	Lines	Key Methods / Notes
QueueEventService	663	CreateQueueEventAsync · BuildDriverListsAsync · OnWindowExpiredAsync · OnShipmentAcceptedAsync · OnDriverPassedAsync · GetActiveEventForDriverAsync · BuildDtoAsync
ShipmentService	346	CreateShipment · ApproveAsync (auto-enqueue + log) · RejectAsync · CancelAsync · SyncStatusFromTripAsync · Search (free-text + status/cargoType/urgent filters)
ShipmentQueueService	272	EnqueueAsync (SignalR broadcast + rebuild lists) · AcceptAsync (FOR UPDATE SKIP LOCKED, 3-retry loop, creates Trip) · PassAsync (cascade to next driver)
TripService	182	Create (auto TRP-YYYY-NNNNNN number) · StatusUpdate + GPS · Terminal states free driver · ResetDriverQueueEntry.HasClaimed · Search by driver
DriverService	204	Onboarding: create Driver + Vehicle · FleetDashboard metrics · SearchDrivers · ■ Hardcoded FleetOwnerId in SaveBasicDetails()
OtpService	186	Random 6-digit OTP · In-memory store (10-min TTL, 5-attempt limit) · MailKit HTML email · Console fallback · Creates User on first sign-in
VehicleService	142	ListByFleetOwnerPhone (LEFT JOIN driver name) · SingleAdd · BulkAdd · DeleteByIdList
SysAdminService	159	Real DB metrics: active drivers, pending shipments, active trips, admin overrides · ForceAssignDriverAsync (bypasses queue, creates Trip directly)
RoleService	70	Normalises role strings (camelCase → snake_case) · Checks user_roles DB join table · Routes to dashboard or onboarding based on profile existence
GoogleAuthService	—	Validates Google ID Tokens via Google.Apis.Auth · Returns email + sub for user lookup
SystemLogService	—	Writes SystemLog rows with EventType, ActorType, EntityType, Metadata (JSONB)
JwtService (singleton)	—	Signs HS256 tokens · 1440-min expiry · Extracts claims in middleware
QueueMaintenanceWorker	76	BackgroundService · PeriodicTimer 15 s · Sweeps expired ShipmentQueueAssignment rows · Calls OnWindowExpiredAsync · Auto-closes QueueEvents past EndTime

11

Data Models

14 EF Core entities — UUID PKs, CHECK constraints, snake_case columns

All entities use **UUID primary keys**. Column names are mapped to snake_case via Fluent API in **AppDbContext.cs** (145 lines, 13 DbSets). Status columns are protected with CHECK constraints; **updated_at** triggers fire on all mutable tables. The custom PostgreSQL type **actor_type_enum** is used in SystemLog for type-safe audit entries.

Model	Key Fields	Purpose
User	Id, Email, Phone, FullName, AuthProvider	Base identity record for all five roles
Driver	UserId, CurrentFleetOwnerId, LicenseNumber, AvailabilityStatus, Verified, TotalTripsCompleted	Driver profile; TotalTripsCompleted drives queue position ordering

Model	Key Fields	Purpose
Vehicle	FleetOwnerId, CurrentDriverId, RegistrationNumber, VehicleType, CapacityTons	Vehicle owned by fleet, assigned to one driver at a time
FleetOwner	UserId, BusinessName, BusinessType, Verified	Business entity owning a fleet of vehicles
Organisation	Name, OrgType, PrimaryContactPhone/Email, PAN, GST, Address	Sender or receiver business entity
Shipment	SenderOrgId, ReceiverOrgId, CargoType/Weight/Volume, Status, AgreedPrice, AdminOverride	Cargo movement request; status flows: pending_approval → approved → dispatched → delivered
Trip	ShipmentId, VehicleId, DriverId, CurrentStatus, GPS coords, Ratings, Financials	Operational execution of a shipment; event-sourced via trip_status_events
ShipmentQueue	ShipmentId, ZoneId, Status (waiting/offered/accepted/expired), CurrentDriverId	Approved shipment awaiting driver claim on the live dispatch board
QueueEvent	StartTime, EndTime, WindowSeconds, Status (live/closed), PriorityRule	Time-slotted access event; only one active event globally at a time
DriverQueueEntry	QueueEventId, DriverId, Position, ClaimWindowStart/End, ShipmentListJson, ClaimableCount	Driver's slot within a queue event; ShipmentListJson is deliberate denormalisation
ShipmentQueueAssignment	QueueEventId, ShipmentQueueId, DriverId, OfferedAt, ExpiresAt, Outcome	Tracks the full offer-routing history for every shipment-driver pairing
DriverShipmentSlot	ShipmentQueueId, ExpiresAt, IsExpired, IsSkipped	One entry in a driver's ShipmentListJson array; mutation state preserved on rebuild
Address	OwnerType, OwnerId, AddressLine1, City, State, Lat/Lng	Polymorphic address attached to an Organisation or Shipment
SystemLog	EventType, ActorType (enum), EntityType, Metadata (JSONB)	Immutable audit trail for all admin actions; metadata stored as JSONB for flexibility

Database Views & Triggers

Name	Type	Purpose
active_trips_view	View	Joins trips + shipments + driver + vehicle for live operations
driver_performance_view	View	Aggregates TotalTripsCompleted, ratings, on-time metrics per driver
vehicle_utilization_view	View	Active vs idle vehicle ratio per fleet owner
trip_analytics_daily	Materialized View	Daily rollup of trip counts, revenue, distance for reporting
updated_at trigger	Trigger	Auto-sets updated_at on INSERT/UPDATE for all mutable tables
trip_status_events	Event-source table	Append-only log of every status transition for a trip
trip_locations	GPS table	Breadcrumb trail of lat/lng coordinates during active trips

1

JSON Column in PostgreSQL

Perf / simplicity

DriverQueueEntry.ShipmentListJson stores each driver's full shipment offer list as a JSON array (JSONB). This deliberate denormalisation avoids N queries per driver poll. The list is rebuilt atomically on every state mutation — expiry, skip, or accept.

2

FOR UPDATE SKIP LOCKED

Concurrency safety

Accept operations use FromSqlRaw to execute `SELECT * FROM shipment_queue WHERE id={0} FOR UPDATE SKIP LOCKED`. This is the correct PostgreSQL pattern for first-come-first-served without blocking other transactions. A 3-attempt retry loop handles the rare case where all rows are locked.

3

Background Worker over DB Triggers

Maintainability

Timer expiry is handled by a 15-second C# BackgroundService rather than PostgreSQL triggers. This keeps business logic (cascading expiry, SignalR broadcasts) in the application layer where it can be unit-tested and reasoned about without stored-procedure complexity.

4

Single Flutter Codebase — Multi-Role UI

Deployment simplicity

All 5 roles share one Flutter app binary with role-based routing via AppRouter (20+ named routes). Post-login, RoleService resolves the role and redirects to the correct dashboard or onboarding screen. Simplifies deployment but includes all role UIs in the binary.

5

Email-First Auth (migrated from Phone OTP)

Security / UX

Migrated from phone OTP to email OTP + Google Sign-In. Phone field kept nullable for backward compatibility. OTP is now real random 6-digit (was hardcoded 123456), stored in-memory with 10-min TTL and 5-attempt rate limit. MailKit SMTP delivers HTML-branded emails.

13

Flutter App Architecture

Presentation → Core → Data · Provider state management · Dio HTTP

The Flutter app (`src/fmp_app/`) is a single-codebase, multi-role mobile application. State is managed with **Provider (ChangeNotifier)**. HTTP calls go through a central **Dio** client at `lib/core/network/api_client.dart` which attaches JWT Bearer tokens via an interceptor and auto-logs out on 401. Session data (email, driverId, token, role) persists across restarts via `flutter_secure_storage`.

Module	Screens / Files	Key Behaviour
auth/	Welcome · EmailInput · OTPVerify · ApprovalPending	Full OTP + Google Sign-In flow; Google ID token sent to backend for validation
role_router/	AccountResolverScreen	Calls RoleService post-login; routes to dashboard or onboarding based on profile existence
onboarding/	RoleSelection · DriverOnboarding · SenderOnboarding · FleetmgrOnboarding	First-time profile creation; sends basic details to respective API endpoints
driver/	Dashboard · Queue · Trips · Profile · Billing · Documents	Queue screen polls <code>/api/queue-events/active</code> every interval; displays countdown timers per slot
sender/	Dashboard · CreateShipment · ShipmentList · Billing · Profile	Shipment CRUD; status-based list filtering; billing summary view
union/	Dashboard · Requests · Queue · QueueEvents · Search · Profile	Approval workflow; admin can create/toggle QueueEvents; search across shipments
fleetmgr/	Dashboard · Drivers · Vehicles · Trips · Documents · Profile	Fleet dashboard with active/idle/issue driver counts; bulk vehicle add via CSV import
sys_admin_dashbo ard/	Tabbed: Metrics · Logs · Users · Shipments	Real DB metrics; system log viewer; user management; force-assign driver to shipment

Key Flutter Dependencies

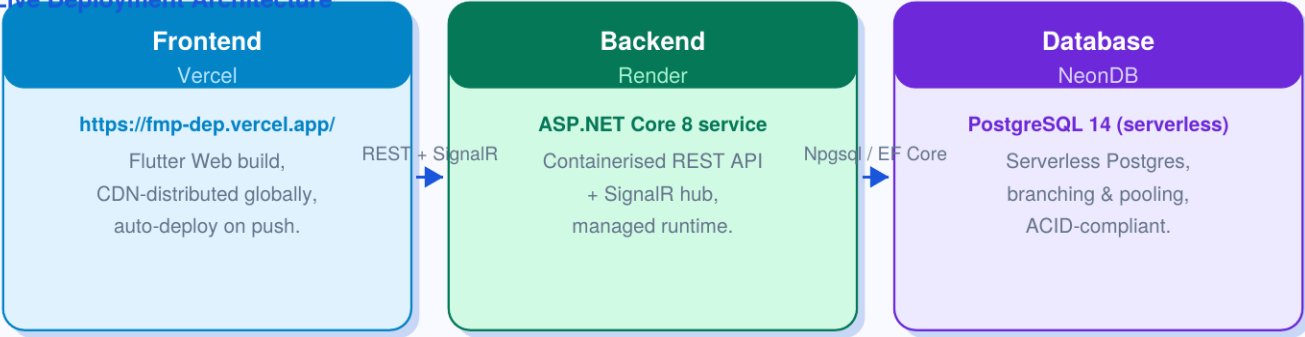
dio	HTTP client with JWT Bearer interceptor and 401 auto-logout
provider	ChangeNotifier state management across all role screens
flutter_secure_storage	Persists session (token, role, email, driverId) across restarts
google_sign_in	Google OAuth flow; ID token forwarded to backend for validation
signalr_netcore	Real-time SignalR connection to ShipmentQueueHub for live queue updates
csv	CSV data import for bulk vehicle addition in Fleet Manager
intl	Date/number/currency formatting across all role screens

14

Deployment

Live Infrastructure — Vercel · Render · NeonDB

Live Deployment Architecture



Layer	Host	Runtime	Endpoint / Notes	Status
Frontend	Vercel	Flutter Web	https://fmp-dep.vercel.app/	Live
Backend	Render	ASP.NET Core 8	Managed container service	Live
Database	NeonDB	PostgreSQL 14 (serverless)	Serverless Postgres w/ branching	Live